

Executive summary

ABC Hub runs eight operational systems on one PostgreSQL database (26 tables, 98,700 rows, ~12 months of history). Reports are slow, departments disagree on numbers, history is hard to analyse and streaming cannot be analysed together with physical rentals.

This project delivers a **layered (medallion) analytical platform on PostgreSQL, fed by an Apache NiFi ETL pipeline**:

- **Bronze** keeps every extracted version of every source row (insert-only history, full lineage): the audit and traceability foundation.
- **Silver** holds validated, cleansed, de-duplicated, conformed entities. All 14 kinds of planted data-quality issue found during profiling are detected and handled, and every correction is flagged on the row.
- **Gold** holds three business star schemas (Customer Daily Activity, Content Monthly Performance, Inventory Daily Utilisation) with conformed, SCD-managed dimensions and a semantic layer of analyst views.

The pipeline is **incremental** (watermark on `updated_at`), **idempotent** (proven: a repeated run changes 0 rows and a full rebuild reproduces every gold table bit-for-bit), fully **audited** (run log, step log, load audit, rejected-record log) and passes **44 automated validation checks** in which every fact measure reconciles exactly to its source.

Question 1: Operational systems and how they interact

1.1 Operational systems identified

The scenario lists eight systems; the ER diagram shows which tables each one owns. Two further data-producing capabilities (streaming playback and the recommendation engine) are implicit in the platform features and generate the largest data volumes.

#	Operational system	Business purpose	Owns (source tables)	Data it generates
1	Customer Management	Registration, profiles, addresses, account status	customer, customer_address, city, country	New customers, profile/status changes, relocations
2	Subscription Management	Streaming plans and each customer's subscription lifecycle	subscription_plan, customer_subscription	Sign-ups, upgrades, cancellations, renewals
3	Content Management	Catalogue of digital and physical titles and their metadata	content, content_type, genre, content_genre, artist, content_artist	New titles, metadata changes
4	Streaming Platform (web/mobile playback)	Delivers digital content on devices	streaming_session, device	One event per viewing session (≈48k/year, the highest volume)
5	Inventory Management	Physical copies held in warehouses	inventory_item, warehouse	Stock additions, condition/status changes, transfers
6	Rental Management	Renting physical copies, due dates, returns, late fees	rental	Rentals, returns, overdue status, late fees
7	Delivery Management	Dispatch, delivery and return logistics via couriers	delivery, courier	Dispatch/delivery/return events
8	Payment Processing	Subscription and rental charges, refunds	payment, payment_method	Payments, failures, refunds
9	Customer Support	Tickets raised by customers	support_ticket, support_category	Tickets, priorities, resolution
10	Engagement & Recommendation	Reviews, wish-lists, personalised recommendations	review, wishlist, recommendation	Ratings, demand signals, recommendation clicks

1.2 High-level interaction diagram

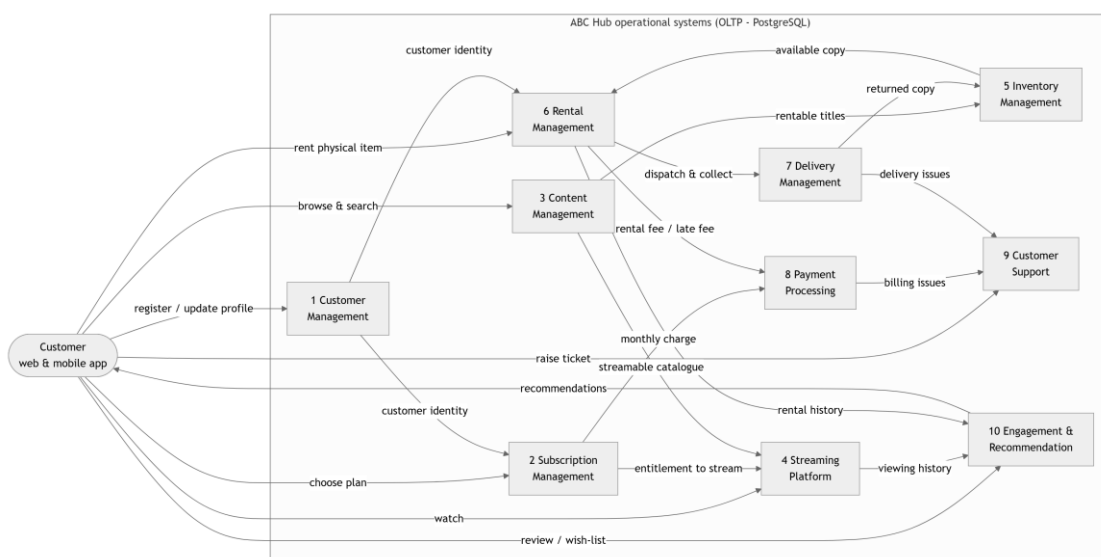


Figure 1: Operational systems and how they interact

How the systems support the business. Customer Management is the master of *who* the customer is; every other system references `customer_id`. Subscription Management entitles a customer to stream and triggers a **monthly payment**. Content Management is the master of *what* is offered: streamable types (Movie, TV Series, Documentary, Music) feed the Streaming Platform, rentable types (Movie, Music, Book, Video Game) feed Inventory Management. Rental Management reserves a physical copy, charges rental and late fees through Payment Processing and hands the copy to Delivery Management, which brings it back into inventory on return. Viewing and rental history feed the Engagement & Recommendation engine, and problems anywhere surface as Customer Support tickets.

Why this matters for analytics. Each system is optimised for its own transactions. A question such as *"revenue per customer across streaming and rentals by country"* needs data from **five** systems (Customer, Subscription, Streaming, Rental, Payment), the root of the reporting problems analysed next.

Question 2: Reporting challenges and recommended solution

2.1 Analysis of the challenges

Challenge reported	Root cause (in the operational set-up)	Consequence
Reports take a long time to prepare	Reports query the normalised OLTP schema directly: 26 tables, many-way joins, row-level scans, competing with live transactions	Slow reports, risk of slowing the customer-facing app
Departments report different figures	No shared definitions: e.g. "revenue" with or without refunds/failed payments, duplicate customers counted twice, ACTIVE vs Active treated as different statuses	Loss of trust; meetings spent reconciling numbers
Historical trends are hard to analyse	OLTP stores current state : a customer's old city or an item's old warehouse is overwritten; no calendar structures	Past reports cannot be reproduced; trends are distorted
Streaming and rentals cannot be analysed together	Each lives in a different system with different grain and keys (stream_id vs rental_id vs payment_id)	No single customer view; cross-selling insight impossible
Business users depend on IT	Only people who know the 26-table model and SQL can build reports	IT bottleneck, slow decisions
(Future) 5-year international growth	Reporting load grows with data; more countries, currencies, time zones	Direct OLTP reporting will not scale

2.2 Recommendation

Build a **dedicated analytical data platform (data warehouse)** separate from the operational systems, organised in **medallion layers** and exposed through **business-friendly dimensional models and a semantic layer** for self-service BI:

1. **Automated ETL (Apache NiFi)** extracts changes from the operational database on a schedule; reporting no longer touches OLTP.
2. **Bronze (raw history)** stores every extracted row version → historical and audit questions become answerable.
3. **Silver (cleansed & conformed)** applies **one** set of validation, cleansing and de-duplication rules → one version of the truth.
4. **Gold (star schemas + pre-aggregated facts)** designed around business processes → fast queries, streaming and rentals joined through **conformed dimensions** (customer, content, date).
5. **Semantic views + BI tools** with agreed KPI definitions → self-service reporting without IT.
6. **Governance**: data-quality tests, run/audit logs, role-based access.

2.3 High-level architecture

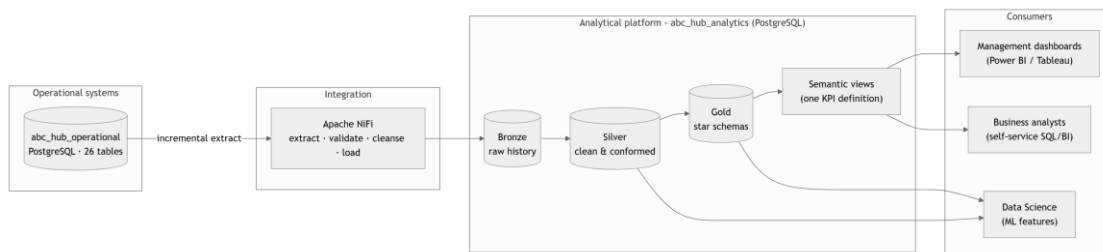


Figure 2: High-level architecture of the recommended solution

Challenge	Solved by
Slow reports	Pre-aggregated gold facts; reporting isolated from OLTP; indexes for analytic access paths
Different figures	Single cleansing rule set in silver; KPI views in gold used by every department
History	Insert-only bronze + SCD Type 2 dimensions + calendar dimension
Streaming + rentals together	Conformed dim_customer / dim_content / dim_date shared by all facts
IT dependency	Denormalised star schemas and labelled views that business users can query directly
Growth	Incremental loads, layered design that can move to a cloud warehouse/lakehouse unchanged

Question 3: Gathering analytical requirements

3.1 Stakeholder questions

Questions are grouped by the areas the brief asks to clarify, then by the six high-level requirements.

Business objectives

1. Which decisions will these analyses support (pricing, content acquisition, inventory purchasing, retention campaigns, expansion)?
2. Which is more important for the next 12 months: growing streaming subscribers or physical rental revenue?
3. How will success of the platform be measured (report preparation time, adoption, trusted numbers)?
4. Which markets are planned for expansion, and must reporting support multiple currencies/time zones?

KPIs and business measures

5. How is **revenue** defined: gross or net of refunds? Are failed payments excluded? Is revenue recognised on payment date or service period?
6. How should subscription revenue be attributed to individual titles (it covers unlimited streaming)?
7. What makes a customer **active**: any login, a stream, a rental, or a payment? Over which window (day / 30 days)?
8. What is a **completed** stream (e.g. $\geq 90\%$ watched)? Does a 2-minute session count as a view?
9. How is **inventory utilisation** defined: days rented $\div$ days owned, or rentals per copy? Are damaged/retired copies included?
10. Is **content popularity** measured by streams, unique viewers, rentals, ratings, wish-lists or a weighted score?
11. What is considered a late return, and should late fees be reported separately?

Required dimensions

12. Should results be analysed by country and city of the customer, by warehouse location, or both?
13. Are subscription plan, device, genre, language, age rating and content type needed as filters?
14. Should customer demographics (age band, gender, registration cohort) be available?
15. Do you need to see a customer's **historical** attributes (e.g. city at the time of a rental) or only the current ones?

Time granularity

16. Daily, weekly, monthly, quarterly or yearly? Is intra-day (hour-of-day) viewing analysis needed?
17. Fiscal or calendar year? Which time zone defines "a day" once the business is international?

Reporting frequency

18. How fresh must data be: real-time, hourly, daily? Which reports are used in daily stand-ups vs monthly board packs?
19. When must the month-end figures be final and frozen?

Historical data requirements

20. How many years must be kept online, and what are the legal retention/audit obligations?
21. Must past reports be reproducible exactly as originally published (point-in-time)?
22. Does Data Science need raw event-level history or aggregates?

Data-quality expectations

23. What should happen to invalid records (reject, fix, flag)? Who owns correcting them in the source?
24. Which fields are mandatory for analysis? What is an acceptable error rate?
25. How should duplicate customers or transactions be identified and resolved?

Performance expectations

26. What is the acceptable response time for dashboards (e.g. < 5 s) and ad-hoc queries?
27. How many concurrent users, and at which peak times?

Security and access

28. Who may see customer PII (name, e-mail, phone, date of birth)? Is masking required?
29. Are there data-residency or privacy regulations (e.g. GDPR-style, Sri Lanka PDPA) per country?
30. Which roles need raw data (auditors, data scientists) vs curated data (analysts, managers)?

Future scalability

31. Expected growth in customers, titles, streams per day and countries over five years?
32. Will new sources be added (mobile app analytics, CRM, marketing, third-party content data)?
33. Is there a preferred cloud provider or on-premise constraint?

Per initial requirement (examples)

Initial requirement	Additional questions
Customer activity	Include inactive/suspended customers? Compare plans? Streaming and rentals together?
Streaming performance	Which device/genre breakdowns? Completion threshold? Peak hours?
Physical rental trends	By warehouse, content type, courier? Late-return rate targets?
Revenue	Net of refunds? Attribution of subscriptions to titles? Currency?
Content popularity	Which measure ranks titles? Ratings weighting? New-release effect?
Inventory utilisation	Definition; retired/damaged copies; target utilisation; re-stocking decisions?
Data Science history	Which features, how far back, event-level or daily, refresh frequency, PII allowed?

3.2 Assumptions (and why they are appropriate)

#	Assumption	Justification
A1	Analysis at daily grain, rolled up to week / month / quarter / year	Matches Scenario A and C grains; any coarser level is derivable from daily, but not vice versa
A2	Data refreshed daily in batch (transactions hourly, gold hourly)	Management reports are daily/monthly; hourly transactional extracts give fresh numbers without real-time complexity
A3	Customer location analysed at country and city of the home address	Available in the source (<code>customer_address</code> → <code>city</code> → <code>country</code>); supports the international-expansion question

#	Assumption	Justification
A4	Customer activity includes streaming, rentals, returns, payments, tickets, reviews and wish-lists	The brief explicitly wants streaming and rentals analysed together; all are customer-initiated events
A5	Revenue = completed payments (sign errors corrected); refunds and failed payments reported separately	Avoids double counting and gives Finance and Marketing the same figure
A6	Title revenue = rental payments + subscription revenue allocated by share of streaming minutes	Subscriptions are not tied to a title (data package note); usage-based allocation is a standard, explainable proxy
A7	A completed stream is $\geq 90\%$ watched; completion is capped at 100 %	Common industry threshold; values $> 100\%$ are data errors
A8	Inventory utilisation = rented item-days $\div$ in-service item-days	Uses time, not counts, so a copy rented once for 30 days outranks one rented twice for a day
A9	History retained indefinitely (bronze insert-only; SCD2 dimensions)	Audit/traceability requirement (Q4); storage is cheap relative to lost history
A10	Customer status and home location change over time and need history (SCD2); name/e-mail corrections do not (SCD1)	Status and location drive segmentation; contact details are corrections
A11	Invalid records are rejected and logged , not silently dropped; fixable issues are corrected and flagged	Keeps analysis complete while making every change auditable
A12	A duplicate customer is the same e-mail under a later customer_id ; the first account survives	Profiling shows 8 such pairs, identical name/phone, a few days apart, with no activity on the copy
A13	Dashboards must respond in < 5 s for a year of data	Achievable with pre-aggregated facts; typical BI expectation
A14	Analysts see no PII ; data scientists see cleansed history; auditors see everything read-only	Least-privilege principle; privacy obligations in multiple countries

3.3 Refined analytical requirements

R1 – Customer Daily Activity. One record per customer per active day with streaming sessions, minutes, distinct titles, completed streams, rentals, returns, late returns, amount spent (subscription / rental split), refunds, failed payments, support tickets, reviews and wish-list additions; analysable by date, customer (demographics, cohort, status, city, country, *as of that day*) and subscription plan *active that day*.

R2 – Content Monthly Performance. One record per title per month with streams, unique viewers, streaming minutes, completion, rentals, unique renters, rental revenue, allocated subscription revenue, total revenue, review count, average rating and wish-list additions; analysable by month, content type, genre, artist, language, age rating and availability (streaming / rental / both).

R3 – Inventory Daily Utilisation. One record per physical copy per day in service with rentals started, returns, rented / available / overdue flags and utilisation; analysable by date, title, content type, warehouse, city and country.

R4 – KPI definitions (single source): Active customers $\cdot$ Streaming hours $\cdot$ Completion rate $\cdot$ Rentals $\cdot$ Late-return rate $\cdot$ Revenue (subscription, rental, total, per active customer) $\cdot$ Refunds $\cdot$ Inventory utilisation % $\cdot$ Average rating $\cdot$ Wish-list demand.

Aspect	Requirement
Time	Daily grain; month via dim_month ; calendar year; 2025-06-06 $\rightarrow$ 2026-06-12 loaded
Frequency	Reference data daily 00:30; transactions hourly (:05); gold build hourly (:30)
History	Unlimited; every source version kept in bronze; SCD2 on customer and inventory item

Aspect	Requirement
Data quality	Mandatory-field validation; standardised codes; sign/range fixes flagged; duplicates rejected and logged; automated reconciliation after every run
Performance	Pre-aggregated facts; dashboard queries < 5 s; incremental loads
Security	Role-based access: analyst (views, no PII), data scientist (silver + gold), auditor (read all), ETL (write)
Scalability	Incremental, idempotent, metadata-driven pipeline; add a table by adding one config entry
Data Science	Cleansed event-level history in silver + daily customer features in gold

Question 4: Modelling for complete history and full traceability

4.1 Recommendation: Data Vault 2.0 for the integration (raw history) layer, dimensional marts on top

The requirement, *retain complete historical information and provide full traceability for auditing and regulatory purposes*, is exactly what **Data Vault 2.0** was designed for. I recommend a Data Vault as the **integration / history layer** of the warehouse, with **Kimball star schemas (SCD Type 2)** built from it for consumption (Q5, Q7).

4.2 Justification

Requirement	How Data Vault 2.0 satisfies it
Complete history	Insert-only : satellites never update or delete; every change is a new row with its load date
Traceability	Every row carries load_dts (when it arrived) and record_source (where it came from); hash keys are deterministic, so a record can be traced from any report back to the source row
Auditability / reproducibility	"What did we know on date X?" = satellites filtered on $\text{load_dts} \leq X$; reports can be re-created exactly as published
Changing sources & growth	New sources/attributes are new satellites or links ; existing structures are never altered (additive schema)
Separation of concerns	Raw vault stores facts <i>as delivered</i> (no business rules); business rules live downstream, so they can be re-applied if they change
Scalability	Hubs, links and satellites load in parallel, no update-in-place; suits large volumes

Alternatives considered

Approach	Why not the primary choice for this requirement
Inmon 3NF EDW	Integrated, but history needs effective dating on every table and schema changes ripple through; harder to audit
Kimball star only (SCD2)	Excellent for analysis, but history is kept only for chosen attributes; business rules are already applied, so raw values cannot be re-derived
One Big Table	Fast to read, but duplicates data and loses change history

4.3 Data Vault model (core of ABC Hub)

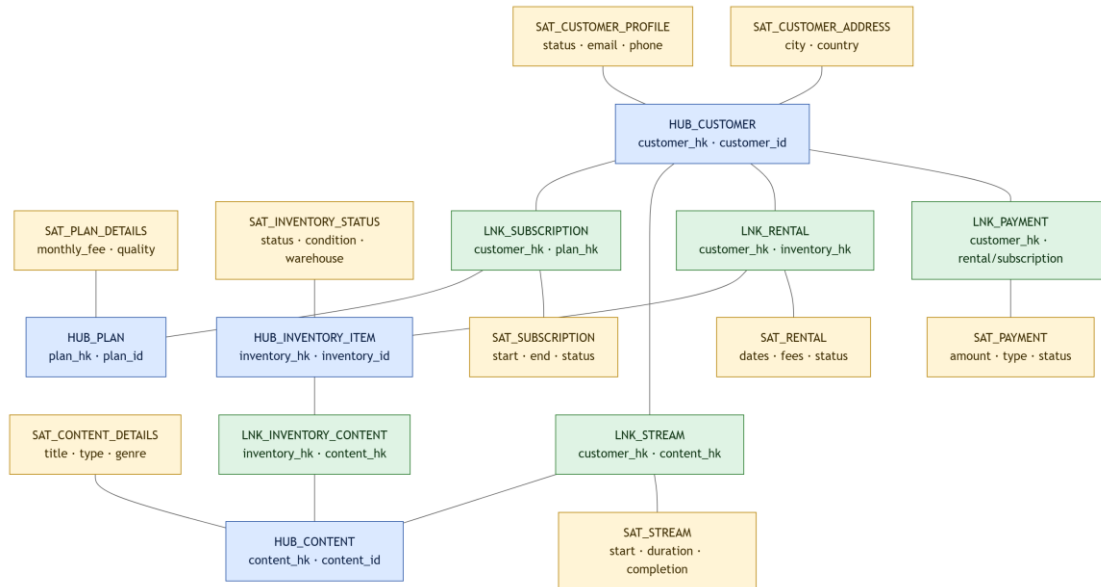


Figure 3: Data Vault 2.0 model for ABC Hub (blue = hub, green = link, yellow = satellite)

Legend. Blue = **hub** (a business key: hash key, business key, load_dts, record_source). Green = **link** (a relationship or transaction between hubs). Yellow = **satellite** (descriptive attributes over time; primary key = parent hash key + load_dts, plus hash_diff for change detection and record_source).

Example structures:

Table	Columns
HUB_CUSTOMER	customer_hk (PK, md5 of customer_id), customer_id, load_dts, record_source
SAT_CUSTOMER_PROFILE	customer_hk (PK, FK), load_dts (PK), hash_diff, status, email, phone, ..., record_source
LNK_RENTAL	rental_hk (PK), customer_hk (FK), inventory_hk (FK), rental_id, load_dts, record_source
SAT_RENTAL	rental_hk (PK, FK), load_dts (PK), hash_diff, rental_date, due_date, return_date, rental_fee, late_fee, status

4.4 How the implemented platform realises these principles

The Mini-Project deliverable (Q8) targets the Q7 star schemas, so a complete raw vault is out of scope. The implementation nevertheless enforces the same traceability guarantees and can be extended into a full vault without re-extracting data:

Data Vault principle	Implemented as
Insert-only history	bronze.<table>: every extracted row version is appended; primary key (source key, updated_at); never updated or deleted
load_dts, record_source	_loaded_at, _record_source on every bronze row
Batch lineage	_batch_id (NiFi batch UUID) on every bronze and silver row; etl.load_audit per batch
Rejected data is never lost	etl.rejected_record keeps the full JSON of each rejected record with the reason
Business-rule transparency	dq_flags on each silver row lists every correction applied
Point-in-time history for analysis	SCD Type 2 dim_customer and dim_inventory_item (effective_from / effective_to / is_current)
Run-level audit	etl.pipeline_run, etl.run_step_log, etl.gold_run_window, etl.gold_change_set

Because bronze already stores every version keyed by business key + load time, hubs/links/satellites can be derived from it at any time (hub = distinct business keys; satellite = bronze rows with `hash_diff`).

Question 5: Simplifying data access for analysts

5.1 Recommendation: dimensional modelling (star schemas) with a semantic layer

Replace direct querying of the 26-table normalised model with **star schemas**: a central **fact table** of measurements at a declared grain, surrounded by **denormalised dimension tables** holding descriptive context. Three pre-aggregated facts (Q7) cover the recurring reports; **analyst views** on top join each fact to its dimensions with business names.

Example. "Monthly revenue by customer country and plan" against the operational schema needs `payment → customer → customer_address → city → country` **plus** `customer_subscription → subscription_plan` with date-range logic and cleansing rules (duplicates, negative amounts, casing) repeated in every report. In the gold layer it is **one** view:

```
SELECT activity_month, home_country, subscription_plan, sum(total_amount_spent)
FROM gold.vw_customer_daily_activity
GROUP BY 1, 2, 3;
```

5.2 How it improves analytical performance

- **Fewer joins:** at most one join per dimension (star join) instead of 5–8 chained joins; the optimiser handles star joins well.
- **Pre-aggregation:** facts are stored at the reporting grain (customer-day, content-month, item-day), so a monthly report scans 7k content rows instead of 48k sessions + 6.5k rentals + 16.9k payments.
- **Small dimensions:** dimensions ($\leq 5k$ rows) fit in memory; filters on attributes hit them, not the facts.
- **Integer surrogate keys and indexes** on every foreign key keep joins cheap.
- **Workload isolation:** heavy analytics run on the warehouse, not the OLTP database used by customers.
- **Cleansing done once** in the pipeline, not in every query.

5.3 Advantages

- **Understandable:** business process in the middle, the "by" words (date, customer, content, warehouse) around it.
- **Consistent numbers:** conformed dimensions + KPI views → every department uses the same definitions.
- **Cross-process analysis:** streaming and rentals share `dim_customer`, `dim_content`, `dim_date`.
- **History:** SCD2 dimensions answer "as it was at the time" questions.
- **Tool-friendly:** Power BI / Tableau expect star schemas; self-service without IT.
- **Extensible:** new facts plug into existing conformed dimensions (bus architecture).

5.4 Limitations and trade-offs

Trade-off	Mitigation in this design
Redundancy / extra storage (denormalised dimensions, aggregates)	Dimensions are small; storage is cheaper than analyst time
ETL complexity (SCD handling, aggregation logic)	Encapsulated in tested, idempotent PostgreSQL functions
Latency: data is as fresh as the last load	Hourly incremental loads; bronze/silver available for near-real-time needs
Aggregates lose detail (e.g. per-session device)	Event-level detail stays in silver for drill-through and Data Science
Non-additive measures (averages, distinct counts, %) can be mis-summed	Stored with their additive parts (rating_sum , rated_review_count , days_rented , days_in_service) and recomputed in views
Design is tied to known questions	Conformed dimensions + silver layer allow new facts to be added quickly

Question 6: End-to-end modern data platform

6.1 Architecture

Treat "it ran without errors" as the start of testing, not the end. Reconciliation, idempotency and incremental tests found bugs that a successful run never showed. I also learned when to let the ETL tool do the work and when to push logic down into the database.

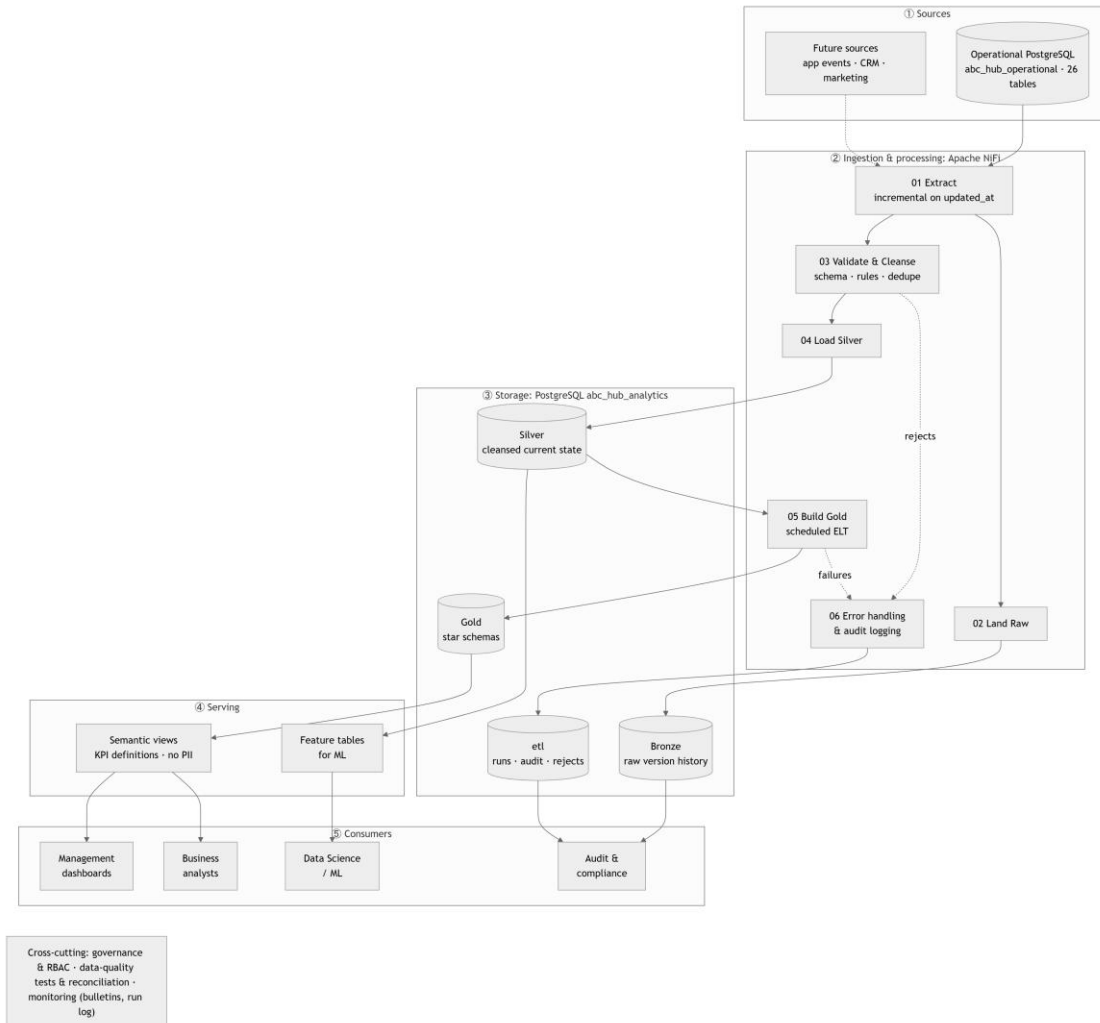


Figure 4: End-to-end modern data platform

6.2 Purpose of each component

Component	Purpose
Operational PostgreSQL	System of record for the 8 business systems; optimised for transactions, not analysis
NiFi – Extract	Pulls only rows whose <code>updated_at</code> exceeds the stored watermark per table; keeps load off the OLTP system
NiFi – Land Raw	Appends every row version to bronze with batch lineage: the audit trail
NiFi – Validate & Cleanse	Enforces mandatory fields/types, standardises codes, fixes sign/range errors, derives missing values, removes duplicates, flags every change
NiFi – Load Silver	Upserts clean entities by primary key
NiFi – Build Gold	Scheduled orchestration of PostgreSQL functions: dimensions first, then facts; run tracking
NiFi – Error handling	Routes every failure/rejection to files, the database reject log and ERROR bulletins

Component	Purpose
Bronze	Immutable history of what the source said and when (traceability, replay, audit)
Silver	Trusted, conformed entities; single place where data-quality rules live; event-level history for Data Science
Gold	Business-oriented star schemas with pre-aggregated facts and SCD dimensions: fast, consistent analytics
etl schema	Control plane: runs, steps, watermarks, change sets, load audit, rejected records
Semantic views	Business names, KPI definitions, no PII: the self-service interface
Data-quality tests	44 automated checks + reconciliation after each run; idempotency and incremental tests
Security	Role-based access (analyst / data scientist / auditor / ETL) with least privilege
Monitoring	NiFi bulletins and logs, <code>etl.pipeline_run</code> status, reject counts

6.3 Data flow (operational → business user)

1. **Extract** – every hour NiFi runs `SELECT ... WHERE updated_at > :watermark` per table (reference tables daily).
2. **Land** – the raw batch is appended to bronze (`INSERT ... ON CONFLICT DO NOTHING`).
3. **Validate / cleanse** – records are validated against an Avro schema (mandatory fields, types), then cleansed with SQL; invalid records and duplicates go to the reject log.
4. **Load silver** – cleansed records are upserted; a trigger stamps `_loaded_at` only if the data really changed.
5. **Build gold** – hourly: find what changed since the last successful build, update dimensions (SCD1/SCD2), rebuild only affected fact rows, advance the watermark.
6. **Serve** – analysts and dashboards query the semantic views; data scientists query silver/gold; auditors trace any figure back through gold → silver → bronze → source row.

6.4 Scalability path (five-year growth)

The same layers map one-to-one onto a cloud lakehouse/warehouse (e.g. object storage + Delta/Iceberg or Snowflake/BigQuery) when volumes outgrow a single PostgreSQL instance; NiFi can run as a cluster; the metadata-driven flow adds a source table with one configuration entry; facts can be partitioned by month; Data Vault can be introduced between bronze and silver for multi-source integration.

Question 7: Analytical dimensional models

7.1 Conformed dimensions (bus matrix)

Dimension	SCD	A – Customer Daily	B – Content Monthly	C – Inventory Daily
dim_date (day)	static	✓	(rolls up to month)	✓
dim_month	static		✓	
dim_customer	Type 2	✓		
dim_subscription_plan	Type 1	✓		
dim_content	Type 1		✓	✓
dim_inventory_item	Type 2			✓
dim_warehouse	Type 1			✓

Design principles applied to all three models:

- **Business-oriented, not a copy of the OLTP model:** 26 operational tables collapse into 7 dimensions; lookup chains (customer_address → city → country, content_genre → genre, content_artist → artist, warehouse → city → country) are flattened into attributes.
- **Surrogate keys** (*_key, integer identity) in every dimension; natural keys (*_id) kept as attributes for traceability; smart keys for time (date_key = YYYYMMDD, month_key = YYYYMM).
- **Unknown member (-1)** in every dimension (and 0 = No active subscription in the plan dimension) so every fact row has a valid foreign key.
- **The primary key of each fact is its grain**, which also prevents duplicate analytical records on reloads.
- **Measures** are numeric, additive where possible; non-additive measures are stored together with their additive components.

7.2 Scenario A: Customer Daily Activity

Grain: one row per **customer** per **calendar day** on which the customer had at least one activity (stream, rental, return, payment, ticket, review, wish-list). Days without activity are not stored (sparse fact); inactivity is derived by comparing with dim_date.

Fact table gold.fact_customer_daily_activity

Column	Type	Key / measure	Why it belongs here
date_key	INT	PK , FK → dim_date	Grain component: day of the activity
customer_key	BIGINT	PK , FK → dim_customer	Grain component: customer version valid that day (SCD2)
subscription_plan_key	INT	FK → dim_subscription_plan	Plan active on that day (0 = none): enables plan comparison
streaming_session_count	INT	measure (additive)	Number of streaming sessions
streaming_minutes	INT	measure (additive)	Total streaming duration
distinct_titles_streamed	INT	measure (non-additive)	Breadth of viewing that day
completed_stream_count	INT	measure (additive)	Sessions ≥ 90 % watched
rental_count	INT	measure (additive)	Physical rentals started

Column	Type	Key / measure	Why it belongs here
returned_item_count	INT	measure (additive)	Items returned
late_return_count	INT	measure (additive)	Returns after due date
total_amount_spent	NUMERIC	measure (additive)	Completed payments (subscription + rental)
subscription_amount_spent / rental_amount_spent	NUMERIC	measure (additive)	Revenue split by stream
refunded_amount, failed_payment_count	NUMERIC / INT	measure (additive)	Payment health
support_ticket_count	INT	measure (additive)	Tickets raised
review_count, wishlist_add_count	INT	measure (additive)	Engagement

Why these are facts: they are **numeric measurements of business events** that are summed, averaged or compared over the dimensions; the descriptive context (who, when, which plan) lives in the dimensions.

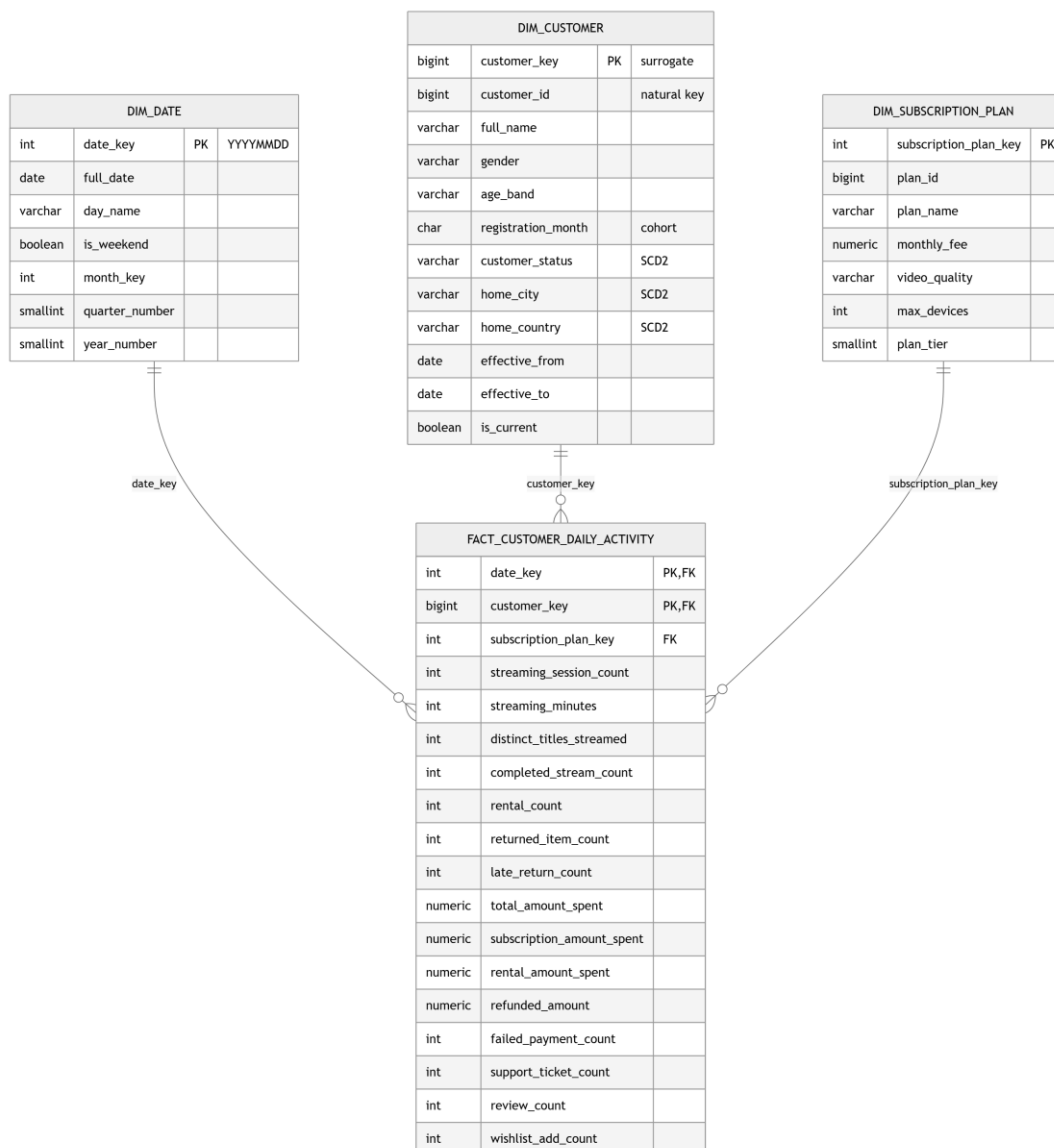


Figure 5: Star schema, Scenario A: Customer Daily Activity

7.3 Scenario B: Content Monthly Performance

Grain: one row per **content item** per **calendar month** in which the title was streamed, rented, paid for, reviewed or wish-listed.

Fact table `gold.fact_content_monthly_performance`

Column	Key / measure	Notes
month_key	PK , FK → dim_month	YYYYMM
content_key	PK , FK → dim_content	
stream_count, streaming_minutes, completed_stream_count	additive	total streams and usage
unique_viewer_count, unique_renter_count	non-additive	distinct customers in the month
avg_completion_pct	non-additive	
rental_count	additive	total rental count
rental_revenue	additive	completed rental payments (rental fee + late fee) attributed via rental → inventory item → content
allocated_subscription_revenue	additive	month's subscription revenue × title's share of streaming minutes (assumption A6)
total_revenue	additive	revenue generated
review_count, rated_review_count, rating_sum	additive	components of the average
avg_customer_rating	non-additive	rating_sum / rated_review_count (invalid 0-ratings excluded)
wishlist_add_count	additive	number of wish-list additions

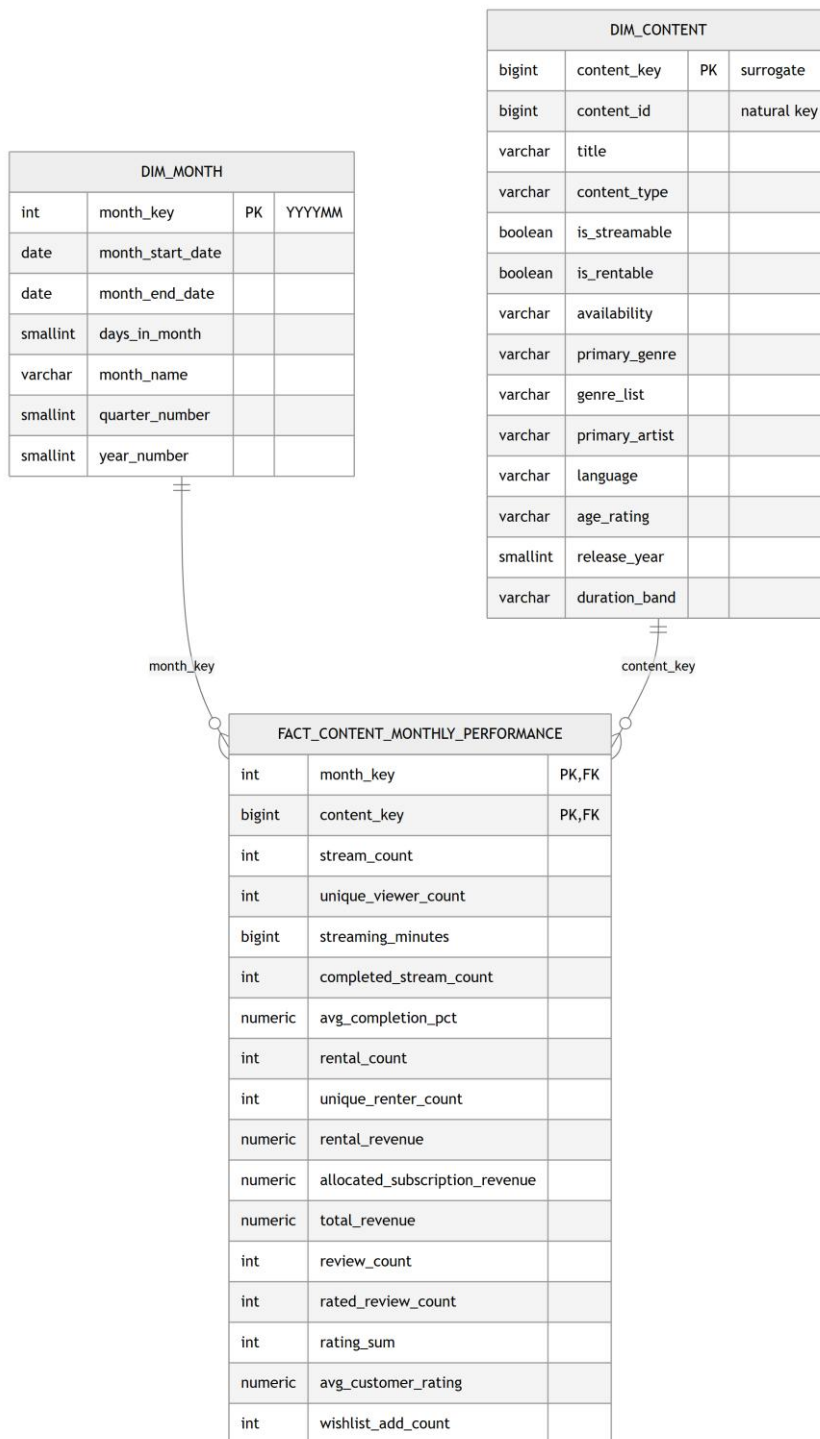


Figure 6: Star schema, Scenario B: Content Monthly Performance

`dim_month` is a **shrunk roll-up dimension** of `dim_date` (same calendar attributes at month level), keeping the monthly fact conformed with the daily facts.

7.4 Scenario C: Inventory Utilisation

Grain: one row per **physical inventory item** per **calendar day** while the item is in service during the snapshot window: a **periodic snapshot** fact. It is deliberately **dense** (idle days included), because "number of days available" must count days on which nothing happened.

Fact table `gold.fact_inventory_daily_snapshot`

Column	Key / measure	Notes
date_key	PK, FK → dim_date	
inventory_key	PK, FK → dim_inventory_item	copy version valid that day (SCD2: status, condition, warehouse)
content_key	FK → dim_content	what the copy is
warehouse_key	FK → dim_warehouse	where the copy was that day
rental_count	additive	number of rentals started
return_count	additive	number of returns
days_in_service	additive over time	always 1 per row
days_rented	additive over time	1 if on loan
days_available	additive over time	number of days available (1 if on the shelf)
is_overdue	semi-additive	on loan past due date
utilisation_pct	non-additive	$100 \times \text{days_rented} / \text{days_in_service}$; roll-ups use the ratio of sums

A check constraint guarantees $\text{days_rented} + \text{days_available} = \text{days_in_service}$ on every row.

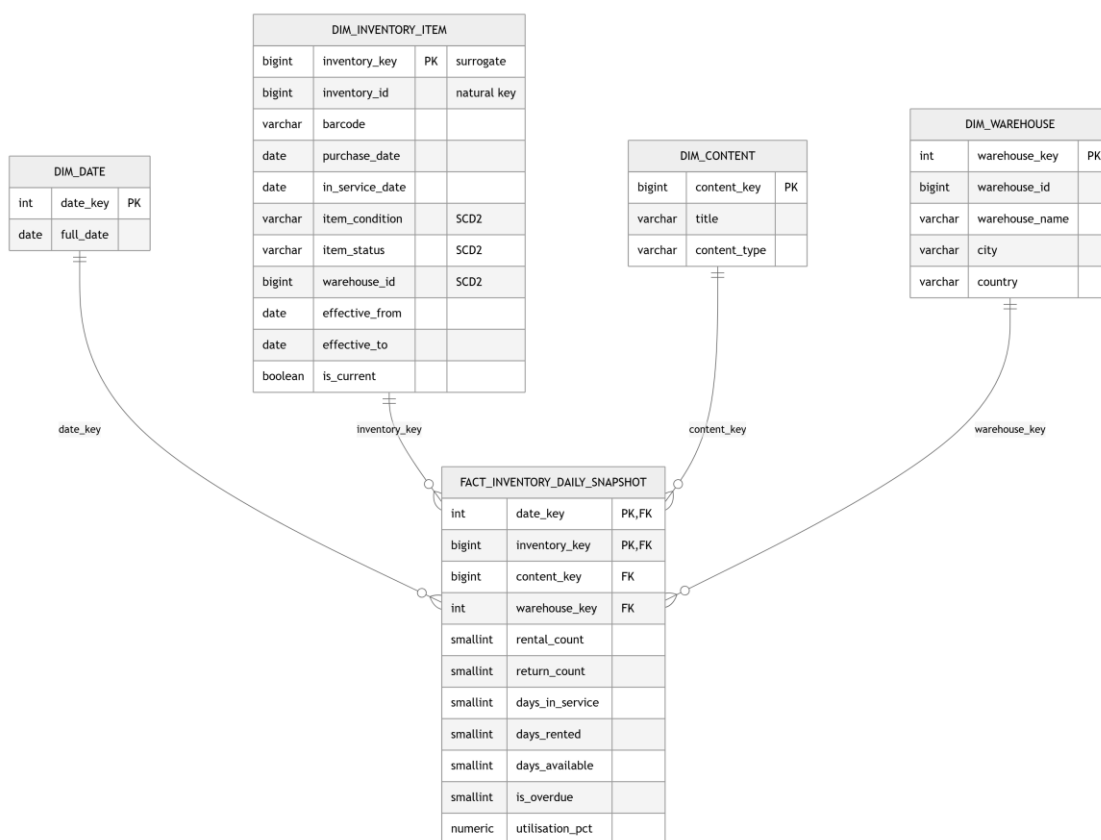


Figure 7: Star schema, Scenario C: Inventory Utilisation

7.5 Key design decisions

Decision	Reason
Separate facts per scenario (not one wide fact)	Each has a different grain; mixing grains causes double counting
Sparse A and B, dense C	A/B record activity; C must count idle days for availability/utilisation
Customer version resolved per event date	Reports show the customer's status/country as it was, not as it is today
Plan in fact A, not in dim_customer	A customer changes plan more often than profile; plan is a separate analysis axis

Decision	Reason
Content type rules (streamable/rentable) in dim_content	Enables "streaming vs rental" comparisons for Movies and Music, which exist in both forms
Store rating_sum + rated_review_count	Averages of averages are wrong; the correct average is recomputed at any level
Revenue split into rental + allocated subscription	Makes the attribution assumption explicit and reversible

Question 8: ETL pipeline with Apache NiFi and PostgreSQL

8.1 Implementation overview

- **Software:** Apache NiFi 2.12.0 (Java 21), PostgreSQL 18.4, PostgreSQL JDBC 42.7.13.
- **Source:** `abc_hub_operational` (provided DDL + 26 CSVs).
- **Target:** `abc_hub_analytics` with schemas `etl`, `bronze`, `silver`, `gold`.
- **Flow as code:** `nifi/build_flow.py` builds the flow through the NiFi REST API from `nifi/flow_config.py` (table metadata) and exports `nifi/ABC_Hub_ETL_Pipeline.json` (NiFi flow definition). 74 processors in 6 process groups.

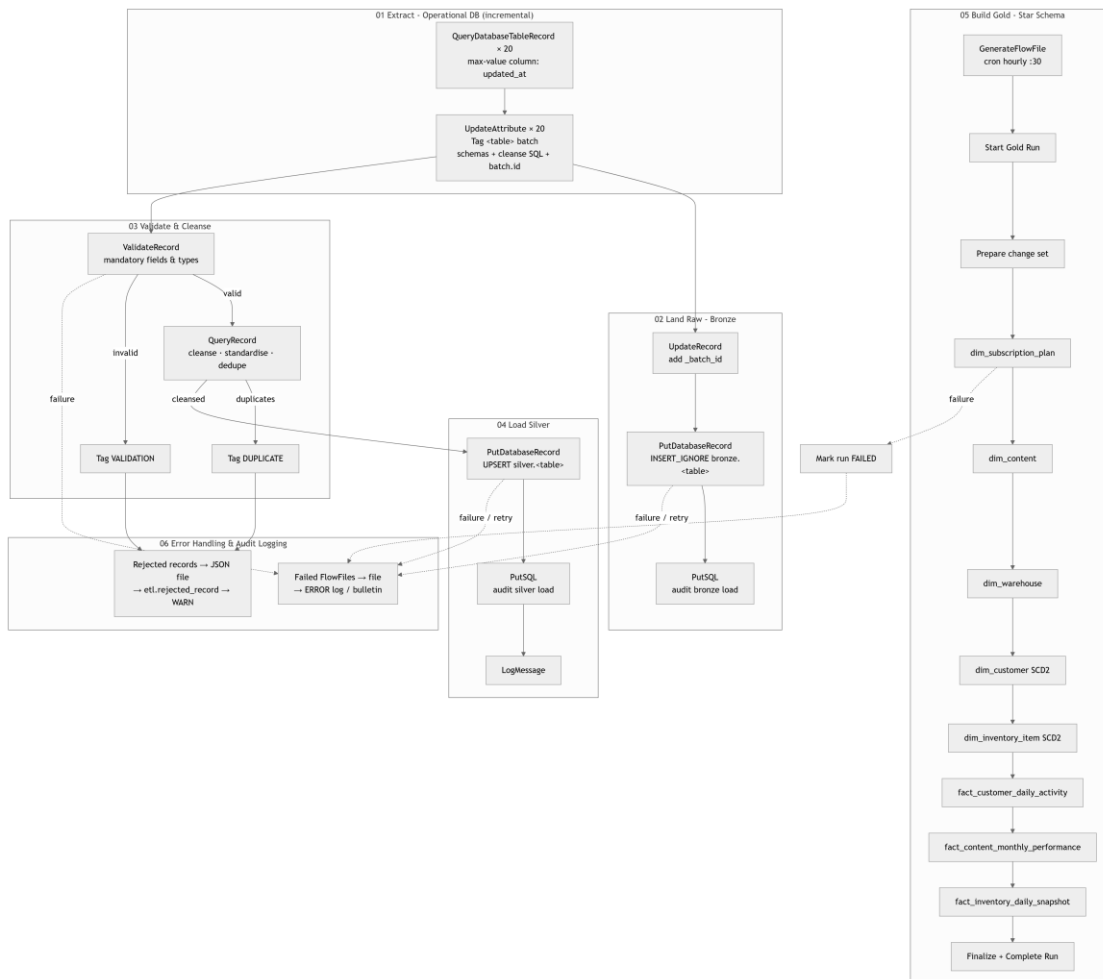


Figure 8: NiFi ETL pipeline (process groups and main processors)

8.2 Task 1: Data extraction

Requirement	Implementation
Connect to the operational DB	<code>DBCConnectionPool</code> "Operational DB Pool (source)"; URL/user/driver path from the parameter context "ABC Hub ETL Parameters"; password is a sensitive parameter (or pgpass); never stored in the flow
Extract the required tables	20 <code>QueryDatabaseTableRecord</code> processors, one per table the models need (device, payment_method, support_category, courier, delivery, recommendation are out of scope)
Read incrementally	<code>Maximum-value Columns = updated_at</code> ; NiFi stores the max value per processor (state) and next time reads only <code>updated_at > stored max</code> : the watermark pattern. First run = full load
Transfer to transformation	Each batch is tagged (<code>UpdateAttribute</code>) with <code>source.table</code> , <code>pk.columns</code> , <code>batch.id</code> , its raw and validation Avro schemas and its cleansing SQL, then leaves through output port "Extracted batches", which feeds both bronze landing and the transformation stage

`updated_at` alone is sufficient because the package guarantees `updated_at ≥ created_at`; the brief's pattern `created_at > wm OR updated_at > wm` is therefore equivalent.

8.3 Task 2: Data transformation

Validation of mandatory fields (`ValidateRecord`). Each table has an Avro schema generated from `flow_config.py` in which analytically mandatory fields (keys, business dates, amounts, statuses) are **non-nullable**. Records with a missing mandatory field or wrong type are routed to `invalid`, tagged `VALIDATION` and written, with NiFi's validation details, to `etl.rejected_record` and a JSON file. *Proven*: a streaming session without `start_time` was rejected with NiFi's message "The following 1 fields were missing: [/start_time]".

Null / invalid values, business transformations (`QueryRecord`). A generated Calcite SQL statement per table standardises and repairs data, and records every repair in `dq_flags`:

Issue found in profiling	Rows	Rule (silver)	Flag
Status / priority / address type casing (ACTIVE, Active, low)	50 + 130 + 42 + 44	INITCAP(TRIM(x))	STATUS_STANDARDISED, ...
E-mail case / spaces	20	LOWER(TRIM(email))	EMAIL_NORMALISED
Missing gender / language	30 / 18	'Unknown'	GENDER_DEFAULTED, LANGUAGE_DEFAULTED
Missing watch duration	480	TIMESTAMPDIFF(MINUTE, start_time, end_time) (exact on every populated row)	WATCH_DURATION_DERIVED
Completion > 100 %	720	cap at 100	COMPLETION_CAPPED
Negative payment amount	170	ABS(amount) (equals the plan fee / rental charge)	AMOUNT_SIGN_FIXED
Negative late fee	65	ABS if returned late / overdue (these were billed), else 0	LATE_FEE_SIGN_FIXED / LATE_FEE_ZEROED
Rating 0 (scale 1–5)	17	NULL (excluded from averages)	RATING_INVALID
Ticket closed before opened	12	closed_date = NULL	CLOSED_BEFORE_OPENED
Duration missing for Books / Video Games	123	legitimate: kept NULL, band "Not applicable"	-

Duplicates. *In-batch*: the cleansing SQL keeps `ROW_NUMBER() = 1` per business key and a second query routes the rest to `duplicates` → tagged `DUPLICATE` → reject log. This removed exactly the 8 re-registered customers, 60 duplicated sessions and 25 duplicated payments found in profiling (93 records, all logged). *Cross-batch*: a copy arriving in a later batch is caught in SQL (`etl.flag_silver_duplicates`) and flagged `is_duplicate` with `duplicate_of_id`; gold ignores it.

Aggregated datasets. The Q7 aggregates are produced by the gold stage (PostgreSQL functions orchestrated by NiFi `ExecuteSQLRecord`). Aggregating inside the database (ELT push-down) is deliberate: an incremental customer-day or item-day aggregate needs the customer's/item's *complete* history, which lives in silver, not in a single NiFi batch. NiFi controls *when* and *in what order* they run and handles failures.

8.4 Task 3: Loading the analytical database

Requirement	Implementation
Dimensions before facts	NiFi group 05 chains <code>ExecuteSQLRecord</code> processors: prepare → 5 dimensions → 3 facts → finalize. A step only starts on the previous step's success
Referential integrity	Real foreign keys from every fact to every dimension; dimension lookups fall back to the -1 <code>Unknown</code> member; test "no fact rows on Unknown members" = 0
No duplicates on repeated runs	Bronze <code>INSERT ... ON CONFLICT DO NOTHING</code> on (key, updated_at); silver <code>UPSERT</code> on the primary key; facts delete-and-reinsert only affected grain keys inside one transaction; fact primary key = grain
Incremental	<code>etl.gold_prepare</code> compares silver <code>_loaded_at</code> with the gold watermark and writes the changed customers / months / items to <code>etl.gold_change_set</code> ; the watermark advances only when the whole run succeeds

Evidence (from `scripts/test_idempotency.sh` and `sql/03_tests/04_incremental_checks.sql`):

- Re-running the whole pipeline with no source changes: every gold step reports **0 rows inserted/updated/deleted**; md5 fingerprints of all gold tables identical.
- Forcing a **full refresh** rebuild: fingerprints identical again.
- Simulating a new business day (17 changed source rows): only **6 of 1,001 customers** and **3 months** re-aggregated; SCD2 versions opened for a suspended customer, a relocated customer and a transferred inventory item, with earlier activity still pointing to the old versions; **22/22** scenario checks pass.

8.5 Task 4: Pipeline management

Requirement	Implementation
Error handling via failure relationships	Every processor's <code>failure</code> (and <code>retry</code> , <code>invalid</code> , <code>unmatched</code>) relationship is connected; database writers <code>retry 3×</code> with back-off before routing to failure; gold failures call <code>etl.fail_run</code> so the run is marked FAILED
Logging of failed records	Group 06: rejected records → JSON file under <code>logs/abc_hub_etl/rejected/<table>/</code> and one row per record in <code>etl.rejected_record</code> + WARN bulletin (the log is idempotent too: a record rejected again only increments <code>occurrence_count</code>); failed FlowFiles → file under <code>logs/abc_hub_etl/failed/<table>/</code> + ERROR log/bulletin
Scheduling	Reference extracts cron <code>0 30 0 * * ?</code> (daily 00:30); transactional extracts <code>0 5 * * * ?</code> (hourly :05); gold build <code>0 30 * * * ?</code> (hourly :30); all downstream processors event-driven (timer 0 s, triggered by queued data); <code>run_pipeline.py</code> / <i>Run Once</i> for on-demand runs
Naming & organisation	Numbered process groups (01–06) in flow order; verb-object processor names ("Extract rental (incremental)", "Upsert into silver.<table>", "Load dim_customer (SCD2)"); labels and comments on groups and processors; funnels and bend points so no connection crosses a component; shared controller services; all environment settings in one parameter context
Audit	<code>etl.load_audit</code> row per batch per layer; <code>etl.pipeline_run</code> + <code>etl.run_step_log</code> per gold run

8.6 Results

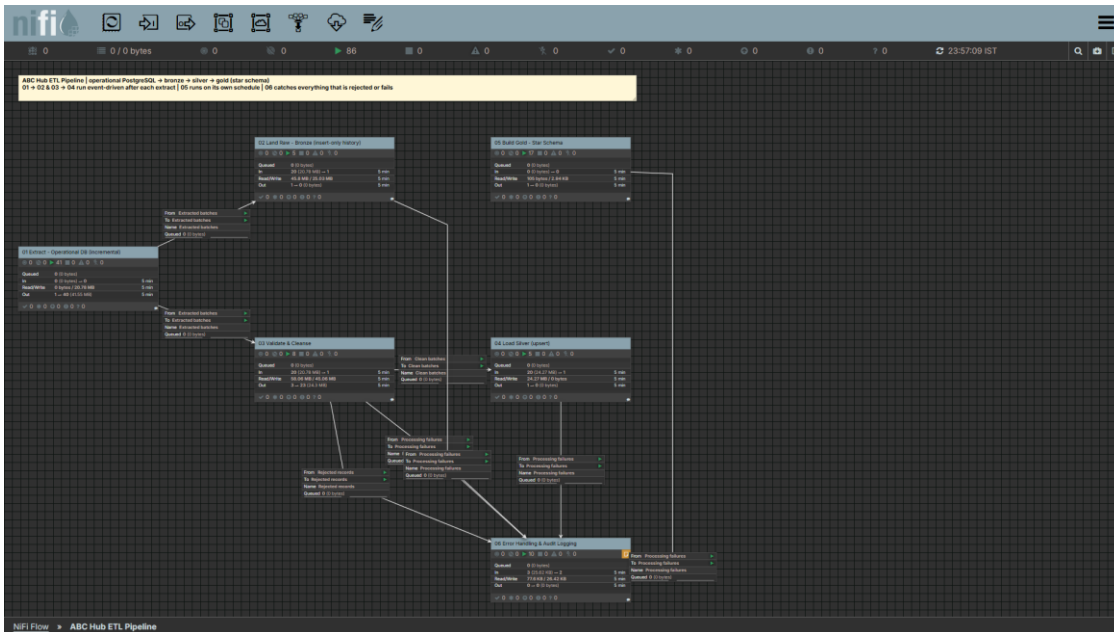
Layer	Rows
Operational source (26 tables)	98,700
Bronze (20 tables, all versions)	87,676
Silver (cleansed, de-duplicated)	87,583
Rejected (in-batch duplicates)	93
<code>fact_customer_daily_activity</code>	68,337
<code>fact_content_monthly_performance</code>	7,099
<code>fact_inventory_daily_snapshot</code>	637,674

End-to-end initial load through NiFi: ≈ 47 s (extract/validate/cleanse/load 12 s, gold 34 s).
Automated validation: **44 / 44 checks pass**, including exact reconciliation of sessions (48,000), minutes (2,782,650), rentals (6,500), returns (6,055), amount spent (175,581.35) and rental revenue (80,048.63) back to silver.

8.7 Screenshots

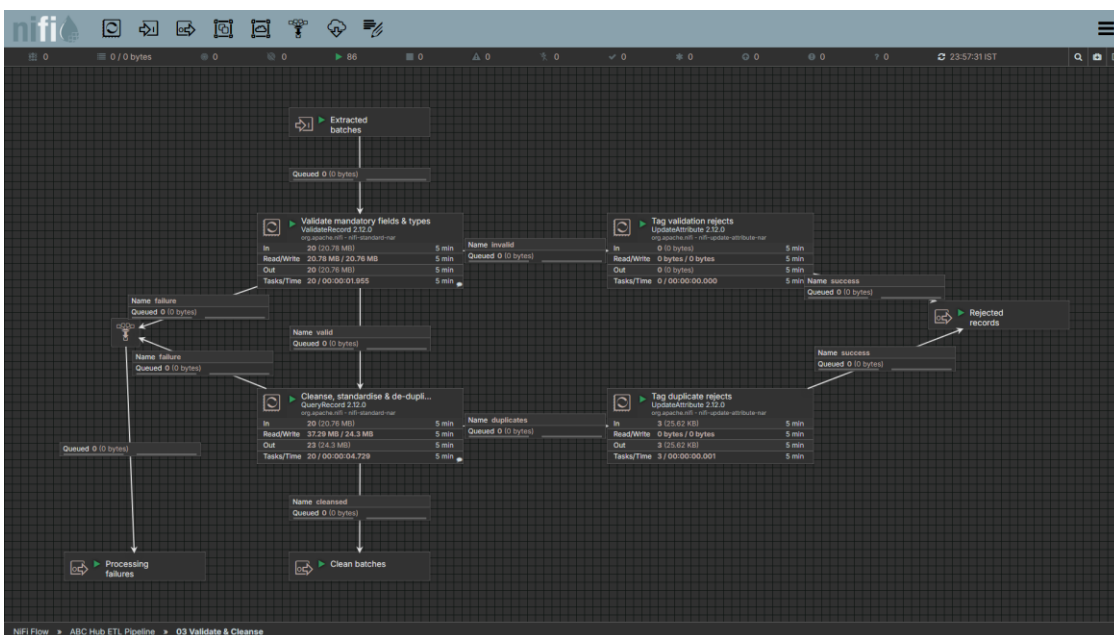
All screenshots are captured from the running system by [scripts/capture_evidence.sh](#); the full set (every process group, run log, populated tables, query results, validation suite) is in [docs/screenshots/](#).

NiFi pipeline overview: six numbered process groups wired through ports:



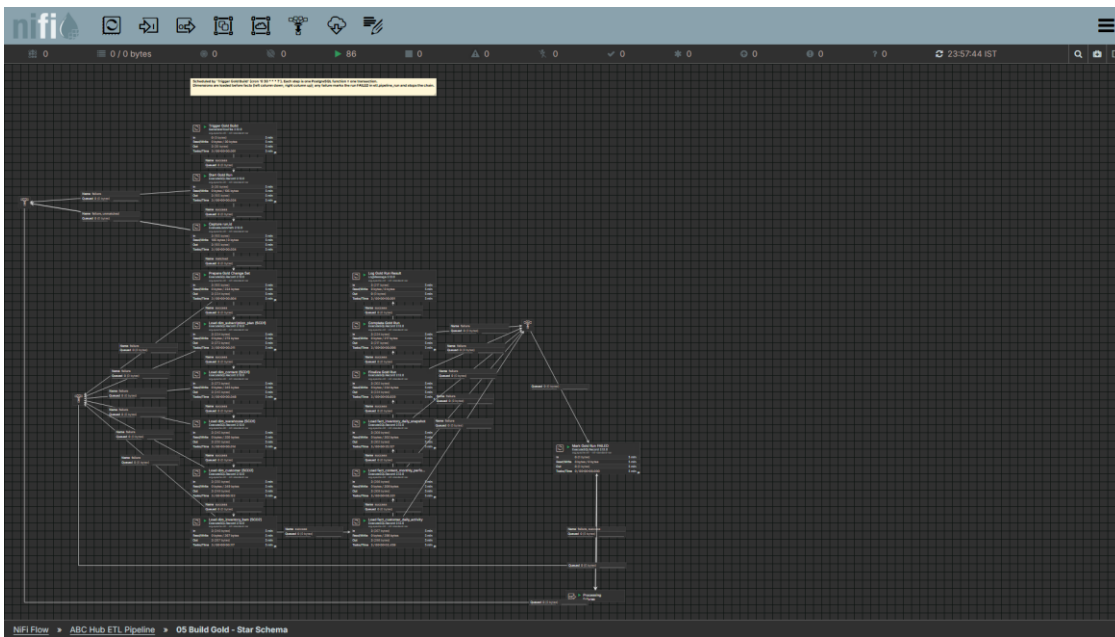
Screenshot: NiFi pipeline overview

03 Validate & Cleanse: `ValidateRecord` $\rightarrow$ `QueryRecord`, with invalid records, duplicates and failures routed to the error handling group:



Screenshot: Validate and cleanse process group

05 Build Gold: dimensions (left column, downwards) before facts (right column, upwards); every failure goes through a funnel to *Mark Gold Run FAILED*:



Screenshot: Build gold process group

Successful execution: NiFi summary after a run (20 batches through every stage):

nifi

Summary

Nifi Summary

Processors

Input Ports

Output Ports

Remote Process Groups

Connections

Process Groups

Filter

Filter by name

Status All Statuses

☐ Primary Node

Filter matched 74 of 74

Name ↑	Type	Process Group	Run Status	In (Size) 5 min	Out (Size) 5 min	Read Write 5 min	Tasks Time 5 min
Add batch lineage field	UpdateRecord	02 Land Raw - Bronze (L...	Running	20 (20.76 MB)	20 (25.03 MB)	20.76 MB 25.03 MB	20 00:00:02.072
Append to bronze <table>	PutDatabaseRecord	02 Land Raw - Bronze (L...	Running	20 (25.03 MB)	20 (25.03 MB)	25.03 MB 0 bytes	20 00:00:05.420
Audit bronze load	PutSQL	02 Land Raw - Bronze (L...	Running	20 (25.03 MB)	0 (0 bytes)	0 bytes 0 bytes	19 00:00:00.192
Audit silver load	PutSQL	04 Load Silver (upsert)	Running	20 (24.27 MB)	20 (24.27 MB)	0 bytes 0 bytes	17 00:00:00.369
Capture run.id	EvaluateJsonPath	05 Build Gold - Star Sch...	Running	3 (105 bytes)	3 (105 bytes)	105 bytes 0 bytes	3 00:00:00.024
Cleanse, standardise & de-duplicate	QueryRecord	03 Validate & Cleanse	Running	20 (20.76 MB)	23 (24.3 MB)	37.29 MB 24.3 MB	20 00:00:04.729
Complete Gold Run	ExecuteSQLRecord	05 Build Gold - Star Sch...	Running	3 (234 bytes)	3 (217 bytes)	0 bytes 217 bytes	3 00:00:00.006
P Extract artist (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (34.43 KB)	0 bytes 34.43 KB	3 00:00:00.286
P Extract city (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (5.27 KB)	0 bytes 5.28 KB	3 00:00:00.290
P Extract content (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (134.56 KB)	0 bytes 134.56 KB	3 00:00:00.067
P Extract content_artist (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (172.44 KB)	0 bytes 172.45 KB	3 00:00:00.114
P Extract content_genre (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (102.87 KB)	0 bytes 102.87 KB	3 00:00:00.435
P Extract content_type (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (705 bytes)	0 bytes 709 bytes	3 00:00:00.031
P Extract country (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (1.99 KB)	0 bytes 1.99 KB	3 00:00:00.021
P Extract customer (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (317.68 KB)	0 bytes 317.69 KB	3 00:00:00.197
P Extract customer_address (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (222.22 KB)	0 bytes 222.22 KB	3 00:00:00.603
P Extract customer_subscription (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (245.94 KB)	0 bytes 245.95 KB	3 00:00:00.427
P Extract genre (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (1.27 KB)	0 bytes 1.27 KB	3 00:00:00.027
P Extract inventory_item (incremental)	QueryDatabaseTableRe...	01 Extract - Operational...	Running	0 (0 bytes)	1 (388.8 KB)	0 bytes 388.8 KB	3 00:00:00.181

Last updated: 23:57:58 IST

1 - 74 of 74

Screenshot: NiFi summary after a run

Run log: the initial NiFi-triggered gold build, dimensions before facts:

Successful ETL execution - run log and step log

proj -d abc_hub_analytics - captured 2026-09-24 23:56:58

etl.pipeline_run (gold builds triggered by Nifi)

run_id	pipeline_name	status	triggered_by	full_refresh	started_at	finished_at	seconds	error_message
1	gold_build	SUCCEEDED	nifi	f	2026-09-24 23:55:05.3552	2026-09-24 23:55:33.985764	28.6	
2	gold_build	SUCCEEDED	nifi	f	2026-09-24 23:55:51.585558	2026-09-24 23:55:52.18471	0.6	
3	gold_build	SUCCEEDED	idempotency-test	t	2026-09-24 23:56:01.02479	2026-09-24 23:56:39.736627	38.7	
4	gold_build	SUCCEEDED	nifi	f	2026-09-24 23:56:50.494868	2026-09-24 23:56:51.220779	0.7	

[4 rows]

etl.run_step_log - initial load triggered by Nifi (dimensions before facts)

step_name	rows_inserted	rows_updated	rows_deleted	seconds	message
gold_prepare	4904	0	0	0.27	window (-infinity, 2026-09-24 23:54:59.646129): 2798 changed keys; snapshot 2025-06-06.2026-06-11; full_refresh=t
load_dim_subscription_plan	4	0	0	0.00	
load_dim_content	600	0	0	0.02	
load_dim_warehouse	10	0	0	0.01	
load_dim_customer	1000	0	0	0.04	
load_dim_inventory_item	1785	0	0	0.04	
load_fact_customer_daily_activity	66337	0	0	2.42	1000 customers rebuilt
load_fact_content_monthly_performance	7099	0	0	0.30	13 months rebuilt
load_fact_inventory_daily_snapshot	637674	0	0	25.08	window 2025-06-06.2026-06-11, 1785 items targeted
gold_finalize	0	1	0	0.31	watermark -> 2026-09-24 23:54:59.646129

[10 rows]

etl.load_audit - batches loaded by Nifi per layer

target_layer	batches	records	first_load	last_load
bronze	20	87676	2026-09-24 23:54:54.141212	2026-09-24 23:54:58.934111
silver	20	87583	2026-09-24 23:54:54.26232	2026-09-24 23:54:59.66167

[2 rows]

Screenshot: ETL run log

Analytical tables populated:

Analytical database populated - rows per layer and table

proj -d abc_hub_analytics - captured 2026-09-24 23:56:58

Row counts per table (bronze = raw history, silver = cleansed, gold = star schema)

layer	table_name	row_count
bronze	artist	250
bronze	city	44
bronze	content	600
bronze	content_artist	1431
bronze	content_genre	1032
bronze	content_type	6
bronze	country	15
bronze	customer	1008
bronze	customer_address	1112
bronze	customer_subscription	1215
bronze	genre	12
bronze	inventory_item	1785
bronze	payment	16867
bronze	rental	6500
bronze	review	3500
bronze	streaming_session	48000
bronze	subscription_plan	4
bronze	support_ticket	1400
bronze	warehouse	10
bronze	wishlist	2800
silver	artist	250
silver	city	44
silver	content	600
silver	content_artist	1431
silver	content_genre	1032
silver	content_type	6
silver	country	15
silver	customer	1000
silver	customer_address	1112
silver	customer_subscription	1215
silver	genre	12
silver	inventory_item	1785
silver	payment	16867
silver	rental	6500
silver	review	3500
silver	streaming_session	48000
silver	subscription_plan	4
silver	support_ticket	1400
silver	warehouse	10
silver	wishlist	2800
gold	dim_content	601
gold	dim_customer	1001
gold	dim_date	4740
gold	dim_inventory_item	1786
gold	dim_month	157
gold	dim_subscription_plan	6
gold	dim_warehouse	11
gold	fact_content_monthly_performance	7099
gold	fact_customer_daily_activity	66337
gold	fact_inventory_daily_snapshot	637674
etl	gold_change_set	5596
etl	gold_run_window	3
etl	load_audit	40
etl	pipeline_run	4
etl	rejected_record	93
etl	run_step_log	30
etl	watermark	1

[57 rows]

Screenshot: Rows per layer

Sample query results:

Sample analytical query results

psql -d abc_hub_analytics - captured 2026-09-24 23:56:59

Monthly business KPIs (gold_vw_monthly_business_kpis)											
month	active_customers	streaming_hours	rentals	late_returns	subscription_revenue	rental_revenue	total_revenue	refunds	inventory_utilisation_pct	revenue_per_active_customer	
2025-06	889	4791.0	441	35	7030.88	10045.01	17875.89	268.49	6.96	20.11	
2025-07	932	4876.1	575	83	8144.60	11645.57	19790.17	789.79	10.25	21.23	
2025-08	920	4672.0	579	97	8207.52	9113.91	17321.43	298.25	12.67	19.83	
2025-09	913	4131.0	552	90	8005.78	8132.77	16138.55	67.82	14.05	17.68	
2025-10	897	3881.4	539	88	7615.07	7910.93	15526.00	69.55	15.25	17.31	
2025-11	894	3671.2	562	91	7632.11	8007.62	15639.73	186.69	17.09	17.49	
2025-12	882	3528.9	552	88	7633.18	7315.76	14948.94	213.01	19.09	16.95	
2026-01	882	3441.9	579	86	7414.40	5455.86	12870.26	57.81	21.08	14.59	
2026-02	843	3087.3	511	71	7203.63	3930.61	11134.24	48.16	22.49	13.21	
2026-03	848	3289.2	524	101	7094.73	3371.40	10466.13	70.91	24.21	12.33	
2026-04	846	3295.7	588	106	6969.97	3053.83	9923.80	140.03	25.56	11.73	
2026-05	811	3608.1	498	103	6683.18	2965.36	8748.54	102.25	27.02	10.79	
2026-06	96	93.8	0	16	5207.67	0.00	5207.67	0.00	23.39	54.25	

(13 rows)

Streaming and rental cross-usage by subscription plan				
subscription_plan	active_customers	streamers	renters	avg_spend
No active subscription	360	0	309	42.10
Basic	348	348	320	109.30
Standard	305	305	287	159.02
Premium	251	251	238	198.75
Family	93	93	87	258.08

(5 rows)

Warehouse utilisation, May 2026 (top 10)						
warehouse_name	warehouse_country	content_type	items_in_service	item_days_rented	item_days_available	utilisation_pct
ABC Hub Warehouse 6	United Kingdom	Video Game	14	197	237	45.39
ABC Hub Warehouse 8	Sri Lanka	Movie	82	970	1572	36.16
ABC Hub Warehouse 9	Canada	Book	33	386	637	37.73
ABC Hub Warehouse 2	Malaysia	Video Game	17	196	331	37.19
ABC Hub Warehouse 1	India	Movie	82	890	1652	35.01
ABC Hub Warehouse 7	United States	Video Game	17	173	327	34.80
ABC Hub Warehouse 10	Australia	Music	44	457	907	33.50
ABC Hub Warehouse 1	India	Music	43	462	1026	31.05
ABC Hub Warehouse 10	Australia	Book	38	365	813	30.88
ABC Hub Warehouse 10	Australia	Video Game	15	144	321	30.87

(10 rows)

Screenshot: Sample query results



Implementation assumptions, limitations and future work

Assumptions

1. `updated_at` is a reliable change indicator (guaranteed by the data package).
2. The 20 extracted tables are those needed by the Q7 models; the other 6 can be added by one entry in `flow_config.py`.
3. The first SCD2 version of every customer/item starts on 1900-01-01 (no earlier history exists).
4. Primary genre = lowest `genre_id`; primary artist chosen by role priority (Director > Lead Actor > Performer > Author > ...).
5. A copy's in-service date is `LEAST(purchase_date, first rental)` because 338 rentals pre-date the recorded purchase date; a Retired copy leaves service after its last return (retirement date not recorded).
6. Overlapping rentals of the same copy (a source issue, 1,436 pairs) count as one rented day, so utilisation never exceeds 100 %.
7. The snapshot window is the span of business dates in silver (2025-06-06 → latest).
1. June 2026 is a partial month (data ends 1 June; returns until 11 June).

Limitations and future work

- A full raw Data Vault between bronze and silver (Q4) for multi-source integration.
- Late-arriving changes to a *past* event date (e.g. a corrected `start_time`) re-aggregate the affected customer but not the old month in fact B; use a full refresh or bronze history to catch these.
- Hard deletes in the source are not detected by a watermark (needs CDC, e.g. Debezium); relevant for Mini Project 02.
- Currency and time-zone dimensions for international expansion.
- A BI dashboard on the semantic views.